TESTES AUTOMATIZADOS EM PROJETO JAVA COM JUNIT 5 E MOCKITO

ATIVIDADE AVALIATIVA -- TESTES UNITÁRIOS COM MOCKITO

NOME:	**TURNO:** Noite	**TURMA:**	**DATA:** 10/11/2026
CURSO: Técnico em Desenvolvimento de Sistemas	**PROF.:**	**HORÁRIO:** 19:00 às 21:40	**TIPO:** Individual

INSTRUÇÕES GERAIS

- Atividade **INDIVIDUAL**, realizada **em sala de aula** no dia **10/11/2026 (terça-feira), das 19:00 às 21:40**.
- Cada aluno deve desenvolver o seu próprio projeto e os seus próprios testes. Não é permitido copiar código de colegas.
- Leia todo o documento antes de começar. As atividades seguem uma ordem crescente de dificuldade.
- Responda às questões escritas neste documento (ou em um arquivo .docx com o seu nome) e implemente os testes no IntelliJ IDEA.
- A entrega deve ser feita **até as 21:40**, no local indicado pelo professor. Entregas após o horário não serão aceitas, salvo orientação do professor.
- Base teórica: **Apresentação 4 -- Mockito** (Aula 04).

1. Objetivo da Atividade

Desenvolver e executar testes unitários para a camada de **Service** de um sistema de produtos e pedidos, utilizando Java, Maven, JUnit 5, **Mockito** e IntelliJ IDEA.

O aluno deverá identificar as dependências de cada classe, substituir o **Repository** real por **mocks**, programar o comportamento desses mocks com `when/thenReturn` e `doThrow`, confirmar as interações com `verify` e registrar os resultados — tudo **sem acessar nenhum banco de dados real**.

2. Cenário do Projeto

A empresa **TechStore Distribuidora** contratou sua equipe para testar o módulo de **cadastro de produtos e pedidos** do seu sistema de vendas. O sistema segue a arquitetura em camadas vista em aula:

Camada	Responsabilidade	No teste com Mockito
Controller	Recebe as requisições HTTP	Não participa do teste unitário
Service	Contém as regras de negócio	**Classe REAL testada** (`@InjectMocks`)
Repository	Acessa o banco de dados	**Substituído por um mock** (`@Mock`)
Banco de Dados	Armazena os dados	**Nunca é acessado**

O banco de dados de produção ainda não está disponível para a equipe de testes. Por isso, os testes deverão isolar o Service usando Mockito. Nesta atividade, o aluno atuará como **analista de testes** e como **desenvolvedor de testes unitários**.

3. Requisitos Funcionais

RF01 -- Cadastro de Produto

- O sistema deve permitir salvar um produto com nome, preço e quantidade em estoque.
- O produto só pode ser enviado ao Repository se todas as validações forem atendidas.

RF02 -- Validação dos Dados do Produto

- O nome não pode ser nulo ou vazio.
- O preço deve ser **maior que zero** (preço 0 ou negativo é inválido).
- A quantidade em estoque não pode ser negativa.
- Dados inválidos devem lançar `IllegalArgumentException` e o método `salvar()` do Repository **não** deve ser chamado.

RF03 -- Busca de Produto por ID

- Quando o produto existir, o Service deve retorná-lo.
- Quando o Repository retornar `null`, o Service deve lançar `ProdutoNaoEncontradoException`.

RF04 -- Tratamento de Falha no Repository

- Se o Repository lançar uma `RuntimeException` (ex.: banco fora do ar), o Service deve lançar `ErroAoSalvarProdutoException`.

RF05 -- Criação de Pedido

- O `PedidoService` deve buscar o produto no `ProdutoRepository`, calcular o **valor total = preço × quantidade** e salvar o pedido no `PedidoRepository`.
- Quantidade do pedido menor ou igual a zero deve lançar `IllegalArgumentException`.
- Produto inexistente deve lançar `ProdutoNaoEncontradoException`.
- Quantidade maior que o estoque deve lançar `EstoqueInsuficienteException`.
- Em qualquer erro, o pedido **não** deve ser salvo.

4. Requisitos Não Funcionais

- RNF01 -- Utilizar Java, Maven, JUnit 5, Mockito e IntelliJ IDEA.
- RNF02 -- A classe de teste deve usar `@ExtendWith(MockitoExtension.class)`.
- RNF03 -- As dependências (Repositories) devem ser mockadas com `@Mock`; a classe testada deve ser criada com `@InjectMocks`.
- RNF04 -- Nenhum teste pode acessar banco de dados, rede ou arquivo real.
- RNF05 -- Cada teste deve validar **um único cenário** e ter nome claro (ex.: `deveLancarExcecaoQuandoNaoEncontrado()`).
- RNF06 -- O aluno deverá apresentar evidências (print) da execução dos testes no IntelliJ IDEA.

5. Técnica de Teste -- Testes Unitários com Mockito

No teste unitário **puro** (como a classe `Calculadora` da Apresentação 3) a classe não depende de nada externo. Já um **Service** depende de um **Repository**, que acessa o banco de dados. Para testar só a regra de negócio do Service, usamos um **mock**: um objeto "de mentira" que imita a dependência real, como um **dublê** em um filme.

Recurso	Para que serve	Exemplo

`@ExtendWith(MockitoExtension.class)`	Liga o Mockito ao JUnit 5	Colocado acima da classe de teste
`@Mock`	Cria um mock da dependência	`@Mock ProdutoRepository repository;`
`@InjectMocks`	Cria a classe REAL testada e injeta os mocks	`@InjectMocks ProdutoService service;`
`when(...).thenReturn(...)`	Programa o que o mock devolve	`when(repository.buscarPorId(1L)).thenReturn(produto);`
`verify(...)`	Confirma que um método do mock foi chamado (interação)	`verify(repository).salvar(produto);`
`never()` / `times(n)`	Confirma que NÃO foi chamado / foi chamado n vezes	`verify(repository, never()).salvar(any());`
`any()`	Aceita qualquer valor do tipo indicado	`any(Produto.class)`
`doThrow(...).when(...)`	Faz o mock lançar uma exceção (também para métodos `void`)	`doThrow(new RuntimeException()).when(repository).salvar(any());`

Lembre-se: `assertEquals` confere um **valor de retorno**; `verify` confere uma **interação** ("esse método foi chamado?"). Um bom teste com Mockito normalmente usa os dois.

6. Código Base para Análise

Crie um projeto Maven chamado **sistema-produtos-mockito** e adicione as dependências abaixo no `pom.xml`:

pom.xml (trecho)

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.11.4</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-junit-jupiter</artifactId>
        <version>5.14.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Produto.java

```
public class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    public Produto(String nome, double preco, int quantidade) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public int getQuantidade() { return quantidade; }
}
```

ProdutoRepository.java -- interface (por isso pode ser substituída por um mock)

```
public interface ProdutoRepository {
    Produto salvar(Produto produto);
    Produto buscarPorId(Long id);
}
```

ProdutoService.java -- classe que será testada

```
public class ProdutoService {

    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }

    public Produto salvar(Produto produto) {
        if (produto == null || produto.getNome() == null
                || produto.getNome().isBlank()) {
            throw new IllegalArgumentException("Nome do produto é obrigatório");
        }
        if (produto.getPreco() <= 0) {
            throw new IllegalArgumentException("Preço deve ser maior que zero");
        }
        if (produto.getQuantidade() < 0) {
            throw new IllegalArgumentException("Quantidade não pode ser negativa");
        }
        try {
            return repository.salvar(produto);
        } catch (RuntimeException e) {
            throw new ErroAoSalvarProdutoException(
                "Falha ao salvar o produto", e);
        }
    }

    public Produto buscarPorId(Long id) {
        Produto produto = repository.buscarPorId(id);
        if (produto == null) {
            throw new ProdutoNaoEncontradoException("Produto não encontrado: " + id);
        }
        return produto;
    }
}
```

Crie também as exceções `ProdutoNaoEncontradoException`, `ErroAoSalvarProdutoException` e `EstoqueInsuficienteException`, todas estendendo `RuntimeException`. Exemplo:

```
public class ProdutoNaoEncontradoException extends RuntimeException {
    public ProdutoNaoEncontradoException(String mensagem) {
        super(mensagem);
    }
}
```

7. Atividade 1 -- Análise do Código

1. Qual é a dependência da classe `ProdutoService`? Como ela é recebida pela classe?
2. Por que o fato de `ProdutoRepository` ser uma **interface** facilita o uso de mocks?
3. No teste, qual objeto deve ser um **mock** e qual objeto deve ser a **classe real**? Justifique.

4. Em quais situações o método `repository.salvar()` **não** deve ser chamado pelo `ProdutoService`?
5. O que o mock devolve se o teste **não** programar o método com `when()`? Que problema isso pode causar no teste de `buscarPorId()`?
6. Por que **não** devemos mockar a própria classe `ProdutoService`?

8. Atividade 2 -- Plano de Teste Unitário com Mocks

Elabore o plano de teste do `ProdutoService`. Use os cenários abaixo como ponto de partida e **complete as colunas vazias**. Acrescente novos cenários se identificar situações não cobertas.

ID	Req.	Cenário	Configuração do mock	Resultado esperado	Verificação (verify)
CT01	RF01	Produto válido ("Mouse", 49.90, 10)	`when(repository.salvar(produto)).thenReturn(produto)`	Retorna o produto salvo	`verify(repository).salvar(produto)`
CT02	RF02	Preço negativo (-10)	Não precisa	`IllegalArgumentException`	`verify(repository, never()).salvar(any())`
CT03	RF02	Preço igual a zero			
CT04	RF02	Nome vazio ("")			
CT05	RF02	Nome nulo			
CT06	RF02	Quantidade negativa (-1)			
CT07	RF02	Quantidade igual a zero		Aceitar (estoque zerado é permitido)	
CT08	RF03	Buscar produto existente (ID 1)	`when(repository.buscarPorId(1L)).thenReturn(produto)`		
CT09	RF03	Buscar produto inexistente (ID 99)		`ProdutoNaoEncontradoException`	`verify(repository).buscarPorId(99L)`
CT10	RF04	Falha de conexão ao salvar	`doThrow(new RuntimeException("Falha na conexão")).when(repository).salvar(any(Produto.class))`		

9. Atividade 3 -- Cenários do Analista de Testes

CT11 -- Produto com preço igual a zero

O Service deve aceitar ou rejeitar? Qual linha do código lança a exceção? Por que usamos `verify(repository, never())` nesse teste?

CT12 -- Produto com preço 0.01

Esse valor está dentro ou fora da regra? Esse cenário é um teste de fronteira? Justifique.

CT13 -- Nome do produto em branco (" ")

O método `isBlank()` retorna true ou false? O mock do Repository chega a ser usado?

CT14 -- Produto não encontrado

O que o mock deve devolver para simular um produto inexistente? Por que o `verify(repository).buscarPorId(99L)` é importante, mesmo com a exceção sendo lançada?

CT15 -- Banco de dados fora do ar

Como simular essa falha **sem** desligar um banco real? Qual exceção o Service deve lançar para quem o chamou?

CT16 -- Objeto específico x any()

Qual a diferença entre `when(repository.salvar(produto))` e `when(repository.salvar(any(Produto.class)))`? Dê um exemplo de quando cada um é mais adequado.

10. Atividade 4 -- Montando a Classe de Teste

Complete as lacunas (_____) para que a classe de teste funcione com JUnit 5 e Mockito. Depois, explique com suas palavras o papel de cada anotação.

ProdutoServiceTest.java -- complete as lacunas

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@_________________(MockitoExtension.class)
class ProdutoServiceTest {

    @_______
    private ProdutoRepository repository;

    @_________________
    private ProdutoService service;
}
```

1. O que acontece se a anotação `@ExtendWith(MockitoExtension.class)` for esquecida?
2. O objeto criado por `@InjectMocks` é um mock ou uma instância real? Justifique.
3. Como o mock `repository` chega até o `service`, se não escrevemos `new ProdutoService(repository)`?

11. Atividade 5 -- Implementação com JUnit 5 e Mockito

Na classe `ProdutoServiceTest`, implemente **todos** os casos de teste do plano (CT01 a CT10). Use os exemplos abaixo como modelo.

Exemplos de testes (baseados na Apresentação 4)

```
@Test
void deveSalvarProdutoComPrecoValido() {
    // Arrange (preparar)
    Produto produto = new Produto("Mouse", 49.90, 10);
    when(repository.salvar(produto)).thenReturn(produto);

    // Act (executar)
    Produto resultado = service.salvar(produto);

    // Assert (verificar)
    assertEquals("Mouse", resultado.getNome());
    verify(repository).salvar(produto);
}
```

```
@Test
void naoDeveSalvarProdutoComPrecoInvalido() {
    Produto produto = new Produto("Mouse", -10, 5);

    assertThrows(IllegalArgumentException.class,
        () -> service.salvar(produto));

    verify(repository, never()).salvar(any());
}

@Test
void deveLancarExcecaoQuandoNaoEncontrado() {
    when(repository.buscarPorId(99L)).thenReturn(null);

    assertThrows(ProdutoNaoEncontradoException.class,
        () -> service.buscarPorId(99L));

    verify(repository).buscarPorId(99L);
}
```

Para o **CT10**, use `doThrow` e verifique se o Service lança `ErroAoSalvarProdutoException`:

```
@Test
void deveLancarErroQuandoRepositoryFalharAoSalvar() {
    Produto produto = new Produto("Teclado", 129.90, 5);
    doThrow(new RuntimeException("Falha na conexão"))
        .when(repository).salvar(any(Produto.class));

    // complete: assertThrows(...) e verify(...)
}
```

12. Atividade 6 -- PedidoService com Duas Dependências

Agora o `PedidoService` depende de **dois** repositórios: o `ProdutoRepository` (para consultar preço e estoque) e o `PedidoRepository` (para salvar o pedido). Observe que `PedidoRepository.salvar()` é um método **void**.

PedidoRepository.java e Pedido.java

```
public interface PedidoRepository {
    void salvar(Pedido pedido);
}

public class Pedido {
    private Produto produto;
    private int quantidade;
    private double valorTotal;

    public Pedido(Produto produto, int quantidade, double valorTotal) {
        this.produto = produto;
        this.quantidade = quantidade;
        this.valorTotal = valorTotal;
    }

    public Produto getProduto() { return produto; }
    public int getQuantidade() { return quantidade; }
    public double getValorTotal() { return valorTotal; }
}
```

PedidoService.java

```
public class PedidoService {

    private final ProdutoRepository produtoRepository;
    private final PedidoRepository pedidoRepository;

    public PedidoService(ProdutoRepository produtoRepository,
                         PedidoRepository pedidoRepository) {
        this.produtoRepository = produtoRepository;
        this.pedidoRepository = pedidoRepository;
    }

    public Pedido criarPedido(Long produtoId, int quantidade) {
        if (quantidade <= 0) {
            throw new IllegalArgumentException(
                "Quantidade deve ser maior que zero");
        }
        Produto produto = produtoRepository.buscarPorId(produtoId);
        if (produto == null) {
            throw new ProdutoNaoEncontradoException(
                "Produto não encontrado: " + produtoId);
        }
        if (quantidade > produto.getQuantidade()) {
            throw new EstoqueInsuficienteException("Estoque insuficiente");
        }
        double total = produto.getPreco() * quantidade;
        Pedido pedido = new Pedido(produto, quantidade, total);
        pedidoRepository.salvar(pedido);
        return pedido;
    }
}
```

Crie a classe `PedidoServiceTest` com **dois** `@Mock` e um `@InjectMocks` e implemente os cenários abaixo:

ID	Cenário	Configuração dos mocks	Resultado esperado
PED01	Pedido válido: Mouse R$ 49,90, estoque 10, quantidade 2	`produtoRepository.buscarPorId(1L)` retorna o produto	Valor total = 99,80 e pedido salvo
PED02	Pedido válido	Igual ao PED01	`verify` dos dois repositórios com `times(1)`
PED03	Quantidade igual a zero	Não precisa	`IllegalArgumentException`
PED04	Quantidade negativa (-3)		`IllegalArgumentException`
PED05	Produto inexistente (ID 99)	`buscarPorId(99L)` retorna `null`	`ProdutoNaoEncontradoException`
PED06	Quantidade maior que o estoque (11 com estoque 10)		`EstoqueInsuficienteException`
PED07	Quantidade igual ao estoque (10 com estoque 10)		Aceitar (valor de fronteira)
PED08	Qualquer cenário de erro (PED03 a PED06)		`verify(pedidoRepository, never()).salvar(any())`

Exemplo de teste para o PED01/PED02

```
@Test
void deveCalcularValorTotalDoPedido() {
    Produto produto = new Produto("Mouse", 49.90, 10);
    when(produtoRepository.buscarPorId(1L)).thenReturn(produto);

    Pedido pedido = service.criarPedido(1L, 2);
```

```
    assertEquals(99.80, pedido.getValorTotal(), 0.001);
    verify(produtoRepository, times(1)).buscarPorId(1L);
    verify(pedidoRepository, times(1)).salvar(any(Pedido.class));
}
```

Dica: para comparar valores `double`, use o terceiro parâmetro do `assertEquals` (tolerância), como `0.001`.

13. Atividade 7 -- Simulando Falhas com doThrow

O método `PedidoRepository.salvar()` é **void**. Por isso, **não** é possível usar `when(...).thenReturn(...)` nele. Para simular uma falha, usamos `doThrow`:

```
doThrow(new RuntimeException("Falha na conexão"))
    .when(pedidoRepository).salvar(any(Pedido.class));
```

Cenário A -- Banco fora do ar ao salvar o pedido

Programe o `doThrow` acima e chame `service.criarPedido(1L, 2)` dentro de um `assertThrows`. Qual exceção é lançada? O `PedidoService` trata esse erro?

Cenário B -- Tentativa de usar when() em método void

Escreva `when(pedidoRepository.salvar(any())).thenReturn(...)`. O código compila? Explique o motivo e qual é a forma correta.

Cenário C -- Falha no ProdutoRepository

Programe `produtoRepository.buscarPorId(1L)` para lançar uma `RuntimeException`. O `pedidoRepository.salvar()` deve ser chamado? Comprove com `verify`.

Cenário D -- Melhorando o Service (desafio)

Altere o `PedidoService` para capturar a falha do `PedidoRepository` e lançar uma exceção própria (por exemplo, `ErroAoSalvarPedidoException`). Escreva um teste que comprove que esse tratamento funciona.

14. Atividade 8 -- Perguntas do Analista de Testes

Responda às perguntas abaixo para **pelo menos três** testes implementados (um de sucesso, um de exceção e um com `doThrow`).

1. Qual requisito está sendo validado pelo teste?
2. Qual método do Service está sendo testado?
3. Qual dependência foi substituída por mock?
4. Como o mock foi programado (`when/thenReturn`, `doThrow` ou nenhum)?
5. Qual é a condição de entrada?
6. Qual é o resultado esperado?
7. Qual é o resultado obtido? O teste foi aprovado ou reprovado?
8. Qual interação foi confirmada com `verify`? Foi usado `never()` ou `times()`?
9. Por que esse teste não precisa de um banco de dados real?

10. O caso é normal, de exceção ou de valor de fronteira?
11. Se o teste falhar, o problema está no Service ou no Repository? Justifique.
12. O teste pode ser executado novamente, de forma independente, com o mesmo resultado?

15. Modelo de Registro de Execução

Preencha um registro para cada teste implementado (copie a tabela quantas vezes for necessário).

ID do Teste	Preencher pelo aluno
Requisito	Preencher pelo aluno
Classe / Método testado	Preencher pelo aluno
Mock(s) utilizado(s)	Preencher pelo aluno
Configuração do mock (when / doThrow)	Preencher pelo aluno
Cenário / Entrada	Preencher pelo aluno
Resultado Esperado	Preencher pelo aluno
Verificação (verify)	Preencher pelo aluno
Resultado Obtido / Status / Observação	Preencher pelo aluno

16. Atividade 9 -- Análise de Cobertura e Boas Práticas

- Todos os requisitos (RF01 a RF05) possuem pelo menos um teste?
- Foi testado o caminho de sucesso e os caminhos de exceção de cada método?
- Os testes de dados inválidos confirmam que o Repository **não** foi chamado (`never()`)?
- Foi testado o retorno `null` do Repository (produto inexistente)?
- Foi simulada uma falha do Repository com `doThrow`?
- Foram testados os valores de fronteira (preço 0 e 0.01; quantidade igual e maior que o estoque)?
- Algum teste mockou a própria classe testada? (isso é um erro!)
- Cada teste programa apenas o comportamento necessário para aquele cenário?
- Os nomes dos testes descrevem claramente o cenário?
- Existe algum cenário relevante ainda não coberto? Justifique.

17. Entrega do Aluno

Entrega **individual**, até as **21:40 do dia 10/11/2026**, contendo:

- Código-fonte do projeto Maven **sistema-produtos-mockito** (compactado em .zip com o nome **NomeSobrenome_Mockito.zip**).
- Classes de modelo, repositórios (interfaces), serviços e exceções.
- Classes de teste `ProdutoServiceTest` e `PedidoServiceTest` com JUnit 5 e Mockito.
- Plano de teste preenchido (Atividade 2) e tabela da Atividade 6.
- Respostas das Atividades 1, 3, 4, 7, 8 e 9 e do Desafio Final.
- Registros de execução (item 15) e evidências (prints) da execução dos testes no IntelliJ IDEA.

18. Critérios de Avaliação

Critério	Pontuação
Análise do código e identificação das dependências (Atividades 1 e 4)	1,5

Elaboração do plano de testes e cenários do analista (Atividades 2 e 3)	2,0
Implementação dos testes do ProdutoService com @Mock, @InjectMocks, when/thenReturn e verify (Atividade 5)	2,5
Testes do PedidoService com dois mocks e simulação de falhas com doThrow (Atividades 6 e 7)	2,0
Registro de execução, análise de cobertura, boas práticas e Desafio Final (Atividades 8, 9 e item 20)	2,0
TOTAL	**10,0**

19. Tecnologias Obrigatórias

- Java (JDK 17 ou superior)
- Maven
- JUnit 5 (junit-jupiter)
- Mockito (mockito-junit-jupiter)
- IntelliJ IDEA

20. Desafio Final -- Análise do Analista de Testes

O analista informa: **"A equipe possui 18 testes automatizados com Mockito e todos estão passando. Portanto, o sistema vai funcionar perfeitamente com o banco de dados de produção."**

Pergunta: essa afirmação está correta? Justifique considerando: o que um mock realmente testa, o isolamento do Service, a diferença entre testar a **regra de negócio** e testar o **Repository real**, os cenários de exceção e de fronteira e a necessidade de outros tipos de teste (por exemplo, testes de integração).

A resposta deve ser técnica e fundamentada, evitando respostas baseadas somente na quantidade de testes executados.